

Watching the NCFED Protocol in Wireshark

A byte-level teardown of NetClaw's N2N Federation channel — how to capture it, how to read it, and the security posture of agent-to-agent traffic on the wire.

TCP/20203

magic = "NCFED"

JSON-RPC 2.0

len+flags framing

proto 053

Subject: live session captured against bgp-daemon-v2 (AS65007 / 7.7.7.7 "Nicholas")

Method: transparent loopback relay + synthesized pcap · **Artifacts:** captures/ncfed-session.pcap, ncfed-decode.log

Reference: automateyournetwork.ca — "NetClaws Assemble: Introducing N2N Federation and the NCFED Protocol"

Classification: Internal / Lab · **Generated:** 2026-07-13

Contents & Executive Summary

This report dissects a real NCFED (NetClaw Federation) session as it appears on the wire, so a network or security engineer can recognize, capture, and reason about agent-to-agent federation traffic using nothing but Wireshark.

1 The NCFED Protocol on the Wire

Transport, port-sharing discrimination, handshake, and the length-prefixed JSON-RPC frame format.

2 Capturing NCFED with Wireshark / tshark

Capture filters, display filters, Follow TCP Stream, and reading the frame boundaries by hand.

3 Annotated Session Walkthrough

Every message from the captured session, with real bytes: handshake → hello → inventory → heartbeat.

4 Security Analysis — Pros & Cons

What the design gets right, and where the wire format exposes risk.

5 What to Look For — Indicators & Red Flags

A field guide to anomalous NCFED traffic for defenders and IDS authors.

TL;DR NCFED multiplexes onto the same TCP port as BGP (179/custom) and NCTUN, discriminated by a 5-byte "NCFED" magic. After an 8-byte AS + router-ID handshake it carries **JSON-RPC 2.0** messages in a trivial 4-byte length + 1-byte flags frame. The good news for defenders: it is **plaintext and self-describing**, so capability descriptors, inventories, and delegated commands are fully visible in Wireshark. The bad news: it is **plaintext** — on an untrusted path there is no transport encryption or message signing, and identity is asserted, not proven.

1 The NCFED Protocol on the Wire

1.1 One port, three protocols

The NetClaw daemon (`bgp-daemon-v2.py`) accepts every inbound connection on a single TCP listener and decides what it is from the **first byte**, then a magic string. This is why a packet capture on that port mixes routing and agent traffic.

First byte(s)	Protocol	Meaning
0xFF ...	BGP-4	Standard BGP marker (16 × 0xFF) — routing peer.
"NCTUN"	NCTUN	NetClaw tunnel channel (data-plane overlay).
"NCFED"	NCFED	Federation control channel — the subject of this report.

The byte 0x4E ('N') is ambiguous, so the listener reads four more bytes to disambiguate NCTUN from NCFED. In Wireshark this shows up as the very first application-data segment after the TCP handshake.

1.2 The 13-byte NCFED handshake

Immediately after the magic, each side asserts its identity: a 4-byte Autonomous System number and a 4-byte IPv4-encoded router-ID. The initiator sends first; the responder replies with its own. No negotiation, no nonce, no signature.

```
0000 4e 43 46 45 44 00 00 fe 4b 0a ff ff 01 initiator → "NCFED" AS65099 10.255.255.1 0000 4e 43
46 45 44 00 00 fd ef 07 07 07 07 responder → "NCFED" AS65007 7.7.7.7 (the daemon)
```

■ bytes 0–4: magic "NCFED" ■ bytes 5–8: AS (uint32 BE) ■ bytes 9–12: router-ID (IPv4)

0x0000fe4b = 65099, 0x0affffff01 → 10.255.255.1; 0x0000fdef = 65007, 07 07 07 07 → 7.7.7.7. The pair as65007-7.7.7.7 is the daemon's **federation identity** — the primary key everything else hangs off.

1.3 The frame format — length + flags + JSON-RPC

Once handshaked, the channel is a stream of frames. Each frame is a fixed 5-byte header followed by a JSON-RPC 2.0 payload:

```
└─ 4 bytes ─┐ └─ 1 byte ┐ LL LL LL LL FF < UTF-8 JSON-RPC payload, LL bytes > | └─ flags: bit0 =
CONTINUATION (message chunked > 64 KB) └─ payload length, uint32 big-endian (struct "!IB")
```

Field	Bytes	Notes
Length	0–3 (u32 BE)	Payload size. Max 65536 (64 KB) per frame; larger messages are split.
Flags	4 (u8)	0x01 = CONTINUATION (more frames follow for this message). 0x00 = final/only.
Heartbeat	00 00 00 00 00	A length-0 frame is a keepalive — sent after 30 s of silence; 3 missed = channel down.

Because it is length-prefixed and not delimiter-based, Wireshark's stream reassembly will happily show you frame boundaries once you know to count 5 + LL bytes forward.

2 Capturing NCFED with Wireshark / tshark

2.1 Grant capture rights without root

On the NetClaw host, give the capture binary the raw-socket capability once, so you never run Wireshark as root:

```
# one-time, persists across reboots
sudo setcap cap_net_raw,cap_net_admin+eip /usr/bin/dumpcap
```

2.2 Capture filter (BPF) — grab only federation traffic

NCFED shares the daemon's listen port, so filter on it. If peers arrive through an ngrok TCP tunnel, capture on the egress interface:

```
# live capture to file, federation port only
tshark -i ens18 -f "tcp port 20203" -w ncfed-live.pcap

# or in the Wireshark GUI — Capture > Options > Capture filter:
tcp port 20203
```

2.3 Display filters — isolate NCFED inside a mixed capture

There is no built-in NCFED dissector, so filter on the magic bytes and the frame structure using `tcp.payload` offsets:

Goal	Display filter
Only NCFED (magic at segment start)	<code>tcp.payload[0:5] == 4e:43:46:45:44</code>
NCFED handshakes only	<code>tcp.payload[0:5] == 4e:43:46:45:44 && tcp.len == 13</code>
Heartbeats (len-0 frames)	<code>tcp.payload == 00:00:00:00:00</code>
Frames carrying JSON (find "jsonrpc")	<code>tcp contains "jsonrpc"</code>
A specific method on the wire	<code>tcp contains "n2n/inventory"</code>
Delegated command / chat traffic	<code>tcp contains "n2n/chat" tcp contains "n2n/tasks"</code>

2.4 Follow the stream

The fastest way to read a session: right-click any NCFED packet → **Follow > TCP Stream**. Wireshark concatenates both directions; the 5-byte binary headers appear as small gaps of unprintable bytes between clean JSON objects. Switch the stream view to **Hex Dump** to see the `LL LL LL LL FF` header preceding each JSON body.

PRO TIP — CUSTOM DISSECTOR A ~40-line Lua post-dissector keyed on `tcp.port == 20203` and the "NCFED" magic will label the handshake fields (AS, router-ID) and split frames on the length prefix, turning the raw stream into a proper protocol tree. Register JSON as the payload subdissector and you get method/params expansion for free.

2.5 How this capture was produced

The reference session was captured without CAP_NET_RAW by interposing a transparent TCP relay between a test peer and the live daemon, recording every byte with timestamps, and synthesizing a valid pcap around the **real payload bytes**. L2/L3 headers are reconstructed loopback TCP; the application layer is genuine daemon output. Conversation summary from the artifact:

Endpoints	127.0.0.1:51823 ↔ 127.0.0.1:20203
Packets	32 (15 ← daemon, 17 → peer)
Bytes on wire	~141 KB (dominated by 2 × 46 KB inventory advertisements)
Duration	43.0 s (long enough to capture a 30 s heartbeat)

3 Annotated Session Walkthrough

The complete captured exchange, in order. Peer as65099-10.255.255.1 (“Byrn”) connects to the live daemon as65007-7.7.7.7 (“Nicholas”).

Pkt	Dir	Type	Message
4	→ daemon	HANDSHAKE	NCFED magic + AS65099 + 10.255.255.1
6	← daemon	HANDSHAKE	NCFED magic + AS65007 + 7.7.7.7
8	→ daemon	REQUEST	n2n/hello id=1
10	← daemon	RESPONSE	capability descriptor (proto 053)
12	← daemon	NOTIFY	n2n/inventory – 46 KB unsolicited advert
14/16	↔	REQ/RES	n2n/consent_state → "federated"
18/20	↔	REQ/RES	n2n/inventory_get → inventory v2 (46 KB)
22	← daemon	NOTIFY	n2n/endpoint_update → 0.tcp.ngrok.io:15091
24	← daemon	HEARTBEAT	len-0 frame @ +30.0 s

3.1 Handshake reply (packet 6)

The daemon proves nothing here — it simply states who it claims to be. Anyone who knows the magic and an AS/router-ID pair can send this.

```
0030 4e 43 46 45 44 00 00 fd ef 07 "NCFED" AS65007 ... 0040 07 07 07 ...7.7.7.7
```

3.2 hello and the capability descriptor (packets 8 → 10)

The peer’s request frame (192 payload bytes) begins with its length prefix, then JSON:

```
00 00 00 bb 00 {"jsonrpc": "2.0", "id": 1, "method": "n2n/hello", "params": {"identity":  
"as65099-10.255.255.1", "display_name": "Byrn", "capabilities": {...}} } ^ 0x000000bb = 187-byte  
payload, flags 0x00 (final)
```

The daemon’s id=1 response reveals its full capability descriptor — everything a peer (or an eavesdropper) needs to know how to talk to it:

```
00 00 01 22 00 {"jsonrpc":"2.0","id":1,"result":{  
"identity":"as65007-7.7.7.7","display_name":"Nicholas", "version":"1.0","capabilities":  
{ "proto_version":"053", "features":["async_tasks","endpoint_reannounce","negotiate"],  
"agent_invoke":"session-id", "reply_shapes":["finalAssistantVisibleText","payloads"]}}
```

3.3 The inventory advertisement (packet 12) — the crown jewels

Because both consents already exist, the daemon **pushes its entire skill/tool inventory unsolicited** — a 46,211-byte frame. This single message enumerates every capability the agent will execute on a federated peer’s behalf. It is the highest-value object in the whole protocol for anyone watching.

```
00 00 b4 83 00 {"jsonrpc":"2.0","id":"as65007-7.7.7.7:1", "method":"n2n/inventory","params":
{"version":1, "skills":[{"name":"aap-automation","description":"Red Hat Ansible Automation
Platform – job template execution ..."}, {"name":"..."}, ... hundreds of entries ... ]}} ^ 0x0000b483 =
46,211 bytes; still < 64 KB so no CONTINUATION flag
```

3.4 endpoint_update — mobility in the clear (packet 22)

A federated peer announces a new public endpoint over the authenticated channel. Useful for NAT/ngrok mobility — but it broadcasts your reachable address in plaintext:

```
00 00 00 93 00 {"jsonrpc":"2.0","id":"as65007-7.7.7.7:2", "method":"n2n/endpoint_update",
"params":{"identity":"as65007-7.7.7.7", "endpoint":"0.tcp.ngrok.io:15091"}}
```

3.5 Heartbeat (packet 24)

Exactly 30.0 s after the daemon's previous transmission, a bare 5-byte frame — the unmistakable NCFED keepalive signature:

```
00 00 00 00 00 length 0, flags 0 – heartbeat, no payload
```

4 Security Analysis — Pros & Cons

NCFED is a control channel for autonomous agents that run skills and tools on each other's behalf. That makes its wire security posture a genuine attack surface, not an academic one.

Strengths

- **Consent-gated by design.** Every method except `n2n/hello` is rejected until the peer is in the federated state. Both sides must have recorded a local and remote grant — presence alone does nothing.
- **Channel-anchored identity.** The AS/router-ID from the handshake anchors the whole session; a peer can only act as the identity it opened the channel with.
- **Port-sharing discrimination is clean.** A bad magic is closed immediately without disturbing BGP — no protocol confusion into the routing plane.
- **Fully auditable / transparent.** Plaintext JSON-RPC means every delegated action is trivially logged, replayed, and inspected. Great for forensics and for an IDS.
- **Bounded frames.** The 64 KB cap and explicit length prefix make it hard to smuggle an unbounded blob; oversized frames are rejected and the channel closed.
- **Heartbeat liveness.** Dead channels self-deregister (3 missed 30 s beats), limiting zombie sessions an attacker could ride.

Weaknesses & Risks

- **No transport encryption.** The reference channel is plaintext TCP. On any untrusted path (incl. the ngrok relay operator's view) inventories, endpoints, and delegated commands are readable.
- **Identity is asserted, not proven.** The handshake carries an AS + router-ID with no signature, key, or nonce. Anyone who learns a federated peer's identity string can attempt to impersonate it toward a daemon that already consented.
- **No message integrity / anti-replay.** No MAC or sequence signing means a captured `n2n/tasks/submit` or `n2n/chat` frame could be replayed or tampered by an on-path attacker.
- **Endpoint update = redirection risk.** `n2n/endpoint_update` is trusted for the channel identity; a spoofed identity could steer future dials to an attacker-controlled host.
- **Inventory disclosure.** The unsolicited 46 KB advert hands an eavesdropper a complete map of every skill/tool the agent can run — ideal target reconnaissance.
- **Consent state is local & soft.** Presence on a channel is treated as remote consent; the trust decision lives in a local SQLite row, not a cryptographic credential.

DEPLOYMENT GUIDANCE Treat NCFED like an unauthenticated management protocol: **never expose it on an untrusted network in the clear.** Wrap it in mutual-TLS or a WireGuard/Tailscale underlay (this host already runs Tailscale — `100.109.82.38`), pin peer identities to keys rather than AS strings, and log every `n2n/tasks` and `n2n/chat` frame for audit. The plaintext design is a forensics gift on a trusted underlay and a liability on the open Internet.

5 What to Look For — Indicators & Red Flags

A field guide for a defender watching NCFED in Wireshark or feeding it to an IDS. Normal federation is quiet and repetitive; abuse looks different on the wire.

5.1 Baseline — what healthy NCFED looks like

- One handshake per channel, then a burst of hello / inventory / consent_state, then long quiet punctuated by **exactly-30 s** heartbeats.
- Traffic is **bidirectional and small** except for periodic inventory advertisements.
- Every frame parses as valid JSON-RPC 2.0 with a known n2n/* method.
- The AS/router-ID in the handshake matches a peer you actually federated with.

5.2 Red flags

Signal on the wire	Why it matters
Handshake AS/router-ID you never federated with	Impersonation attempt or rogue agent probing. Cross-check against the federation_peer table.
n2n/tasks or n2n/chat before a federated handshake	Protocol-state violation — the daemon rejects these pre-federation; seeing them means someone is probing the method surface.
Bursts of n2n/inventory_get	Reconnaissance — an attacker enumerating your skill/tool catalog for a target list.
Unexpected n2n/endpoint_update	Redirection attempt — steering future dials to a hostile host. Verify the new endpoint.
Frame length header that never matches payload size	Malformed framing / fuzzing / desync attack against the reassembler.
CONTINUATION flag (0x01) with frames summing near/over limits	Possible attempt to exhaust the 64 KB reassembly buffer or smuggle a large payload.
Duplicate identical task/chat frames	Replay attack (no anti-replay in the protocol) — same bytes, new timing.
Missing heartbeats then a reconnect from a new source IP	Session takeover after a dropped channel — legitimate for ngrok churn, but verify the identity re-handshake.
Plaintext NCFED on a WAN / non-VPN interface	The protocol should ride a trusted underlay; on the open wire the whole exchange is exposed.

5.3 tshark one-liners for triage

```
# every JSON-RPC method seen, ranked — spot recon & unknown methods
tshark -r ncfed.pcap -Y 'tcp contains "method"' \
  -T fields -e text 2>/dev/null | grep -oE '"n2n/[a-z/_]+"' | sort | uniq -c

# list all NCFED handshakes and the identities claimed
tshark -r ncfed.pcap -Y 'tcp.payload[0:5]==4e:43:46:45:44 && tcp.len==13' \
  -T fields -e ip.src -e tcp.payload
```

```
# flag heartbeat gaps > 35s (possible dropped/hijacked channel)
tshark -r ncfed.pcap -Y 'tcp.payload==00:00:00:00:00' -T fields -e frame.time_relative
```

DETECTION SUMMARY Because NCFED is self-describing plaintext, a defender has an unusually strong position: you can see exactly what one agent asked another to do. Build detections on (1) identity mismatch at handshake, (2) method-before-federation, (3) inventory-enumeration bursts, and (4) endpoint-update redirection — and alert on any NCFED frame that appears on an untrusted interface at all.

Artifacts referenced: `captures/ncfed-session.pcap` (32 pkts, 141 KB) and `captures/ncfed-decode.log`. Payload bytes are genuine output of the live `bgp-daemon-v2` instance (AS65007 / 7.7.7.7); Ethernet/IP/TCP headers were synthesized around the recorded application stream. Protocol constants (magic, 64 KB frame cap, 30 s / 3-miss heartbeat, port discrimination) are drawn from `bgp/constants.py`, `bgp/agent.py`, and `bgp/federation/` in the `netclaw` repository. Reference article: automateyournetwork.ca — “NetClaws Assemble: Introducing N2N Federation and the NCFED Protocol.”